

DOKUMENTACJA PROJEKTU

14. MediaPlayer

Przedmiot: Zaawansowane aplikacje internetowe

Autor projektu	Dominik Czubiński
Numer albumu	30363
Rok akademicki	2025/26
Repozytorium GitHub	https://github.com/marekpilarz493-hue/MediaPlayer

Spis treści

1. Wstęp i cel projektu
2. Rozwiązywany problem i zakres funkcjonalny
3. Założenia i ograniczenia
4. Zastosowany stack technologiczny
5. Architektura i struktura katalogów
6. Opis implementacji
7. Format danych i import/eksport
8. Instalacja i uruchomienie
9. Wdrożenie produkcyjne
10. Osadzanie playera na istniejącej stronie
11. Testy
12. Zrzuty ekranu
13. Możliwe rozwinięcia
14. Podsumowanie

1. Wstęp i cel projektu

W ramach projektu przygotowałem webowy odtwarzacz materiałów wideo. Od początku chciałem, żeby można było użyć go na dwa sposoby: jako samodzielnej strony oraz jako komponentu dołączanego do już istniejącej strony WWW. Główną część rozwiązania napisałem w postaci klasy JavaScript MediaPlayer, która tworzy cały interfejs odtwarzacza w wybranym elemencie DOM.

Nie próbowałem odwzorowywać całego serwisu w rodzaju YouTube, bo nie taki był cel projektu. Skupiłem się na wymaganych funkcjach: kolejce odtwarzania, sekcjach filmu, komentarzach do sekcji, osi czasu oraz imporcie i eksporcie opisów. W wersji samodzielnej dodałem również upload pliku na serwer i możliwość dodania filmu po adresie URL. W trybie serwerowym przesłany materiał mogę automatycznie przekonwertować przez FFmpeg do MP4 H.264/AAC.

2. Rozwiązywany problem i zakres funkcjonalny

W projekcie skupiłem się na problemie prostego odtwarzania filmu razem z opisem jego struktury. Użytkownik może zaznaczyć konkretne fragmenty materiału, przypisać do nich komentarze, a później przenieść cały opis do innej przeglądarki lub instalacji za pomocą pliku JSON.

2.1. Funkcje użytkownika

- odtwarzanie pliku wideo za pomocą elementu HTML5 <video>,
- dodawanie wielu filmów do kolejki i przechodzenie między nimi,
- automatyczne przejście do następnego filmu po zakończeniu bieżącego,
- usuwanie filmu z kolejki,
- tworzenie sekcji czasowej z nazwą, początkiem i końcem,
- ustawienie początku lub końca sekcji na aktualny czas filmu,
- dodawanie i usuwanie komentarzy przypisanych do sekcji,
- wyświetlanie sekcji na proporcjonalnej osi czasu,
- kliknięcie osi czasu lub sekcji w celu zmiany czasu filmu,
- import i eksport opisu do JSON,
- lokalne zapamiętywanie konfiguracji w localStorage,
- upload filmu na serwer, automatyczna konwersja do MP4 H.264/AAC oraz dodanie filmu po URL lub ścieżce,
- uruchomienie playera jako osobnej strony lub osadzonego komponentu.

3. Założenia i ograniczenia

Projekt celowo zrobiłem w dość prostej formie, odpowiedniej do projektu studenckiego. Nie dodawałem bazy danych ani systemu logowania, bo nie były potrzebne do spełnienia założeń. Opis playlisty zapisuję w pamięci przeglądarki albo w pliku JSON, a przesłane filmy przechowuję jako zwykłe pliki na dysku serwera. Dzięki temu kod jest czytelny i mogę łatwo prześledzić działanie każdej części aplikacji.

- brak kont użytkowników i uprawnień do poszczególnych playlist,
- brak bazy danych i trwałego serwerowego zapisu sekcji/komentarzy,
- brak streamingu adaptacyjnego HLS/DASH i wyboru wielu jakości; upload jest jedynie normalizowany przez FFmpeg do jednego formatu MP4 H.264/AAC,
- zewnętrzny URL filmu musi być możliwy do odtworzenia przez przeglądarkę,
- walidacja JSON jest podstawowa i sprawdza przede wszystkim obecność playlisty,
- upload ma limit 500 MB i obsługuje zestaw popularnych rozszerzeń, m.in. MP4, AVI, MKV, DivX/Xvid, MOV, WebM, WMV i MPEG,
- interfejs nie zawiera rozbudowanej edycji istniejącej sekcji - można ją usunąć i utworzyć ponownie.

4. Zastosowany stack technologiczny

Warstwa	Technologia	Zastosowanie	Uwagi
Frontend	HTML5	Struktura strony i element <video>	Bez systemu szablonów
Frontend	CSS3	Układ, responsywność, wygląd playera	Osobne style komponentu i strony
Frontend	JavaScript ES6	Logika playera i obsługa interakcji	Bez frameworka frontendowego
Backend	Node.js	Uruchomienie serwera	Skrypt server.js
Backend	Express 4	HTTP i pliki statyczne	Prosty serwer projektu
Backend	Multer 2	Odbiór uploadu plików wideo	Zapis tymczasowy do temp-uploads
Dane	JSON	Import i eksport opisów	Czytelny format tekstowy
Dane	localStorage	Zapamiętanie opisu w przeglądarce	Nie przechowuje samego filmu
Backend / multimedia	FFmpeg	Konwersja formatów i kodeków do MP4 H.264/AAC	Program systemowy uruchamiany przez child_process.spawn()

5. Architektura i struktura katalogów

Projekt podzieliłem na kilka prostych części, żeby logika playera, obsługa strony i backend nie mieszały się w jednym pliku. Sam komponent oddzieliłem od kodu strony samodzielnej i od serwera. Dzięki temu plik media-player.js mogę dołączyć także do innej strony, bez kopiowania całej aplikacji demonstracyjnej.

```
MediaPlayer-project/  
|-- server.js  
|-- package.json  
|-- temp-uploads/  
|-- README.md  
|-- public/  
|   |-- index.html  
|   |-- embed-demo.html  
|   |-- assets/  
|       |-- css/player.css  
|       |-- css/app.css  
|       |-- js/media-player.js  
|       |-- js/app.js  
|   |-- data/sample-description.json  
|   |-- media/sample.mp4  
|   |-- uploads/  
|-- docs/
```

5.1. Podział odpowiedzialności

- `media-player.js` - komponent i logika odtwarzania,
- `app.js` - formularze strony standalone, localStorage i komunikacja z uploadem,
- `server.js` - serwer HTTP, upload i uruchamianie konwersji FFmpeg,
- `player.css` - style potrzebne samemu komponentowi,
- `app.css` - wygląd dodatkowego panelu strony głównej,
- `embed-demo.html` - przykład użycia komponentu na innej stronie.

6. Opis implementacji

6.1. Klasa MediaPlayer

Najważniejszą częścią projektu jest napisana przeze mnie klasa MediaPlayer. Konstruktor przyjmuje element lub selektor oraz opcje wejściowe. Następnie tworzę interfejs, podłączam obsługę zdarzeń i wczytuję dane początkowe.

```
const player = new MediaPlayer('#player', {
  playlist: [
    {
      title: 'Mój film',
      src: '/media/film.mp4',
      sections: []
    }
  ]
});
```

Dane playera przechowuję w obiekcie `this.data`, a `this.currentIndex` wskazuje aktualnie wybrany film. Widok kolejki, sekcji i osi czasu odświeżam osobnymi metodami renderującymi. Taki podział był dla mnie czytelniejszy niż umieszczanie całej logiki w jednym długim fragmencie kodu.

6.2. Kolejka odtwarzania

Kolejkę odtwarzania obsłużyłem kilkoma prostymi metodami. `addMedia()` dodaje film do tablicy playlisty, a `loadMedia(index)` ustawia jego adres w elemencie ``. Przyciski Poprzedni i Następny korzystają z indeksu bieżącej pozycji. Dodatkowo po zdarzeniu `ended` automatycznie wywołuję `next()`, dzięki czemu po zakończeniu filmu może rozpocząć się następny element kolejki.

6.3. Sekcje i komentarze

Każdą sekcję zapisuję jako obiekt z nazwą, czasem początku, czasem końca i tablicą komentarzy. Przy dodawaniu sprawdzam, czy koniec sekcji jest późniejszy niż początek oraz czy podane czasy nie wykraczają poza długość filmu. Po dodaniu sortuję sekcje według czasu rozpoczęcia.

```
{
  title: 'Omówienie',
  start: 12.5,
  end: 30,
  comments: ['Ważny fragment filmu.']
}
```

6.4. Oś czasu

Po załadowaniu metadanych filmu mam dostęp do `video.duration`. Na tej podstawie wyliczam położenie sekcji procentowo względem całej długości materiału. Przykładowo sekcja od 20 do 40 sekundy w filmie trwającym 100 sekund zaczyna się na 20% szerokości osi i zajmuje kolejne 20%.

```
left = section.start / duration * 100%
width = (section.end - section.start) / duration * 100%
```

Marker bieżącego czasu aktualizuję przy zdarzeniu `timeupdate`. Kliknięcie w oś czasu przeliczam na pozycję względem jej szerokości, a następnie ustawiam odpowiednią wartość `video.currentTime`. Dzięki temu użytkownik może przejść do wybranego fragmentu bez korzystania wyłącznie z domyślnego paska przeglądarki.

6.5. Delegacja zdarzeń i własne zdarzenie zmiany

Dla przycisków tworzonych dynamicznie zastosowałem prostą delegację zdarzeń. Elementy mają atrybut `data-action`, a jeden listener sprawdza, jaką akcję należy wykonać. Nie muszę dzięki temu podpinąć osobnego listenera do każdego nowego przycisku po każdym renderowaniu widoku.

Po zmianie danych emituję własne zdarzenie `CustomEvent("mediaplayer:change")`. Strona samodzielna nasłuchuje go i zapisuje aktualny opis w localStorage. Celowo zrobiłem to w ten sposób, żeby sam komponent MediaPlayer nie był na stałe związany z jednym sposobem zapisywania danych.

6.6. Upload plików

Upload plików zrobiłem po stronie serwera. Formularz wysyła film przez `fetch()` jako `FormData` do endpointu `POST /api/upload`. Multer zapisuje oryginalny plik w katalogu `temp-uploads`, a następnie z poziomu Node.js uruchamiam FFmpeg przez `child\_process.spawn()`. Film konwertuję do MP4 z obrazem H.264 (`libx264`) i dźwiękiem AAC. Gotowy plik trafia do `public/uploads`, a plik tymczasowy usuwam. Backend zwraca względny adres gotowego MP4 i dodaje go do playlisty. Dzięki temu nie muszę liczyć na bezpośrednią obsługę AVI, MKV czy DivX/Xvid przez przeglądarkę.

Przy tej części projektu ważne było dla mnie rozróżnienie kontenera od kodeka. MP4, MKV i AVI są kontenerami, natomiast H.264, DivX i Xvid to kodeki wideo. Sama końcówka pliku nie gwarantuje więc, że przeglądarka go odtworzy. Dlatego przy uploadzie sprowadzam materiał do jednego przewidywalnego zestawu: MP4 + H.264 + AAC.

```
const response = await fetch('/api/upload', {
  method: 'POST',
  body: formData
});
```

7. Format danych i import/eksport

W eksportowanym pliku JSON zapisuję wersję formatu i playlistę. Każdy film ma tytuł, adres źródła i listę sekcji, a każda sekcja przechowuje czasy oraz komentarze. Do eksportu dodaję też pole `exportedAt` z datą utworzenia pliku, ale nie jest ono potrzebne podczas importowania danych.

```
{
  "version": 1,
  "playlist": [
    {
      "title": "Film demonstracyjny",
      "src": "/media/sample.mp4",
      "sections": [
        {
          "title": "Początek",
          "start": 0,
          "end": 4,
          "comments": ["Przykładowy komentarz"]
        }
      ]
    }
  ]
}
```

Eksport zrobiłem przy użyciu ``Blob`` i tymczasowego adresu URL tworzonego w przeglądarce. Przy imporcie odczytuję zawartość pliku metodą `file.text()`, wykonuję `JSON.parse()`, a następnie przekazuję gotowe dane do `loadDescription()`.

8. Instalacja i uruchomienie

8.1. Wymagania

- Node.js 18 lub nowszy,
- npm,
- FFmpeg dostępny w systemie (polecenie `ffmpeg -version`` powinno działać),
- współczesna przeglądarka internetowa (Chrome, Edge, Firefox itp.).

8.2. Uruchomienie lokalne

1. Aby uruchomić projekt lokalnie, najpierw rozpakowuję go i przechodzę do katalogu `MediaPlayer-project``.
2. Następnie sprawdzam instalację FFmpeg poleceniem `ffmpeg -version``.
3. Zależności projektu instaluję poleceniem `npm install``.
4. Serwer uruchamiam poleceniem `npm start``.
5. Po uruchomieniu serwera otwieram w przeglądarce `http://localhost:3000``.
6. Wersję osadzaną mogę sprawdzić pod adresem `http://localhost:3000/embed-demo.html``.

```
npm install
npm start
```

Zależności backendu zapisałem w pliku `package.json``, dlatego po pobraniu projektu wystarczy wykonać `npm install``. Nie ma potrzeby dołączania katalogu `node_modules``, ponieważ npm może go odtworzyć. FFmpeg traktuję osobno, ponieważ jest programem zainstalowanym w systemie i nie znajduje się w `node_modules``.

9. Wdrożenie produkcyjne

Najprostszy wariant wdrożenia wymaga hostingu obsługującego Node.js oraz zainstalowanego FFmpeg. Po przesłaniu kodu na serwer instaluję zależności i uruchamiam `server.js``. Port mogę ustawić zmienną środowiskową `PORT``, a jeżeli FFmpeg znajduje się w niestandardowym miejscu, jego ścieżkę mogę przekazać przez `FFMPEG_PATH``.

```
npm install --omit=dev
PORT=3000 npm start
```

Przy typowym wdrożeniu aplikację można wystawić za reverse proxy, na przykład Nginx, które przekazuje ruch HTTPS do procesu Node.js. Katalogi `public/uploads`` i `temp-uploads`` muszą mieć prawa zapisu dla procesu serwera. W większym systemie wybrałbym osobną usługę do przechowywania plików lub object storage, ale w tym projekcie zapis na dysku jest wystarczający i prostszy do pokazania.

Jeżeli film jest już wcześniej wgrany na hosting w formacie obsługiwanym przez przeglądarkę, nie muszę korzystać z endpointu uploadu. Wystarczy podać jego bezpośredni adres w formularzu. Przy zewnętrznych źródłach nadal obowiązują ograniczenia kodeków i zasady dostępu ustawione na serwerze z plikiem. Automatyczna konwersja dotyczy materiałów wysyłanych przez endpoint uploadu.

10. Osadzanie playera na istniejącej stronie

Komponent celowo oddzieliłem od strony demonstracyjnej. Do osadzenia playera na istniejącej stronie potrzebuję tylko plików `player.css` i `media-player.js` oraz elementu HTML, w którym utworzę player. `app.js` nie jest potrzebny, ponieważ obsługuje formularze i dodatkowy panel mojej strony samodzielnej.

```
<link rel="stylesheet" href="/assets/css/player.css">
<div id="my-player"></div>
<script src="/assets/js/media-player.js"></script>
<script>
  new MediaPlayer('#my-player', {
    playlist: [
      { title: 'Mój film', src: '/video/film.mp4', sections: [] }
    ]
  });
</script>
```

Pełny przykład osadzenia dodałem w `public/embed-demo.html`. Jeżeli komponent miałby działać na innej domenie lub w innym katalogu, wystarczy odpowiednio zmienić adresy do CSS, JavaScriptu i pliku video.

11. Testy

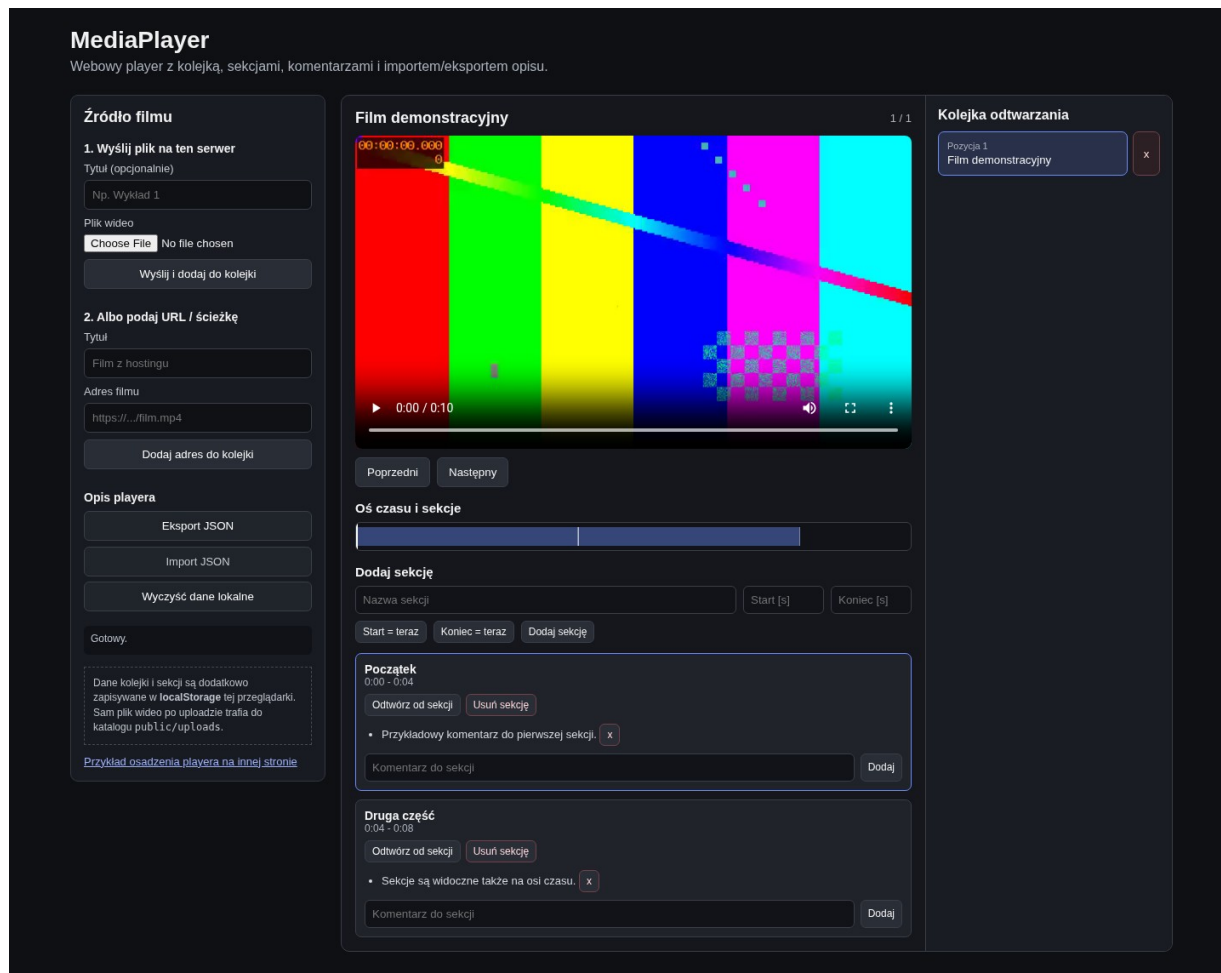
Podczas przygotowania projektu sprawdziłem najważniejsze funkcje frontendu w przeglądarce oraz składnię plików JavaScript. Testowałem między innymi start playera, zmianę filmu, komentarze, eksport danych i wersję osadzaną. Osobno sprawdziłem też mechanizm konwersji plików AVI/Xvid i MKV do MP4.

Scenariusz	Oczekiwany rezultat	Wynik
Start playera	Film demonstracyjny, kolejka i sekcje są widoczne	OK
Metadane filmu	Długość filmu = 10 s, oś czasu działa	OK
Komentarz	Nowy komentarz pojawia się w sekcji	OK
Zmiana filmu	Tytuł i źródło zmieniają się	OK
Usunięcie z kolejki	Pozycja znika i stan jest aktualizowany	OK
Eksport	Dane zawierają version, exportedAt i playlist	OK
Wersja embed	Komponent działa bez app.js	OK
Składnia JS	node --check nie zgłasza błędów	OK
Konwersja AVI/Xvid	Upload tworzy MP4 H.264/AAC w public/uploads	OK

Konwersja MKV	Plik wynikowy MP4 odtwarza się w HTML5 <video>	OK
---------------	--	----

Przed uruchomieniem testu uploadu na innym komputerze trzeba wykonać `npm install` i sprawdzić `ffmpeg -version`. Szczególnie istotny jest test AVI/Xvid i MKV, ponieważ wtedy można potwierdzić, że backend rzeczywiście tworzy MP4 i że wynikowy plik odtwarza się w elemencie HTML5 ``.

12. Zrzuty ekranu



Rys. 1. Samodzielna strona MediaPlayer z kolejką, sekcjami i osią czasu.

Ten dokument udaje zwykłą stronę, do której dołączono komponent MediaPlayer przez CSS i JavaScript.



- baza danych SQLite/PostgreSQL do zapisu playlist, sekcji i komentarzy,
- system kont i uprawnień,
- API REST do zapisu danych po stronie serwera,
- edycja istniejących sekcji i przeciąganie ich granic na osi czasu,
- zmiana kolejności filmów metodą drag & drop,
- miniatury filmów i sekcji,
- napisy, HLS/DASH, kilka jakości odtwarzania i kolejka zadań transkodowania dla dużych filmów,

- przechowywanie plików w zewnętrznej usłudze, np. S3,
- bardziej rozbudowana walidacja importowanego JSON oraz testy automatyczne.

14. Podsumowanie

W projekcie zrealizowałem wymagany zakres webowego MediaPlayera. Aplikacja obsługuje kolejkę filmów, sekcje, komentarze i oś czasu, a przygotowany opis można przenosić za pomocą JSON. W wersji samodzielnej dodałem też upload, adres URL filmu i konwersję wielu popularnych formatów do MP4 H.264/AAC. Oddzielenie klasy `MediaPlayer` pozwala mi użyć tego samego komponentu również na istniejącej stronie.

Celowo wybrałem niewielki i czytelny stack: HTML, CSS i JavaScript po stronie klienta oraz Node.js z Express i Multer po stronie serwera. Do normalizacji filmów backend uruchamia FFmpeg. Nie dodawałem dużego frameworka ani bazy danych, ponieważ w tym projekcie ważniejsze było dla mnie czytelne pokazanie działania playera i poszczególnych funkcji niż rozbudowywanie aplikacji o elementy, które nie były wymagane.